Snehal -- Backend API + LLM Streaming

Built the FastAPI backend with Groq LLM streaming and REST API layer

1. FastAPI Application Setup (main.py)

Created the FastAPI application with a lifespan context manager that pre-warms the embedding model and ChromaDB connection on startup. This eliminates cold-start latency -- the first user query is as fast as every subsequent one.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    loop = asyncio.get_event_loop()
    await loop.run_in_executor(None, _get_embed_model)  # loads multilingual-e5-small
    await loop.run_in_executor(None, _get_collection)   # connects ChromaDB
    yield

app = FastAPI(title="LexAid API", version="0.1.0", lifespan=lifespan)
```

2. CORS Middleware

Configured Cross-Origin Resource Sharing so the Next.js frontend on localhost:3000 can call the FastAPI backend on localhost:8000 without being blocked by the browser same-origin policy.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

3. SSE Streaming Endpoint -- /api/query

The core endpoint. Instead of waiting for the full LLM answer, it streams tokens back to the frontend as Server-Sent Events (SSE), giving the real-time typewriter effect. Three SSE event types are sent in order: "sources" (act names, sent first so badges appear instantly), "token" (each word from Groq), "done" (final chunk scores).

```
@app.post("/api/query")
def query(req: QueryRequest):
    def generate():
        yield from stream_answer(req.question, category=req.category)
    return StreamingResponse(
        generate(),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"}
    )
```

4. Groq LLM Integration (rag.py)

Integrated Groq cloud LLM using the official groq Python SDK. Primary model is gpt-oss-120b with automatic fallback to gpt-oss-20b if the primary fails. Temperature set to 0.1 for consistent, factual legal answers. max_tokens=2048 to prevent response truncation mid-sentence.

```
stream = groq.chat.completions.create(
    model="openai/gpt-oss-120b",
```

```
    messages=messages,
    temperature=0.1,
    max_tokens=2048,
    stream=True,
)
for chunk in stream:
    delta = chunk.choices[0].delta.content
    if delta:
        yield f'data: {{"type":"token","text":{delta!r}}}\n\n'
```

5. System Prompt Engineering

Designed the LLM system prompt with strict grounding rules to prevent hallucination and ensure every answer cites actual statute text.

- Answer ONLY from statutory sections provided -- no outside knowledge
- Cite Act name + section number for every legal point made
- Use plain language -- explain legal jargon when it must be used
- If retrieved sections do not answer the question, say so explicitly
- Always end with the standard legal disclaimer on its own line

6. Additional API Endpoints

/api/acts -- Returns all 17 acts from ACT_MAP dynamically (no hardcoding in frontend)

/api/retrieve -- Debug endpoint: returns raw ChromaDB chunks without LLM (used for testing retrieval quality)

/health -- Status check used by deployment health monitors

Suraj -- Data Ingestion + Vector Database

Collected and ingested 17 Indian statutes into ChromaDB with 2,255 chunks

1. PDF Collection & ACT_MAP Setup

Collected 17 official Indian statute PDFs covering criminal law, consumer rights, labour rights, property, RTI, civil rights, disability, and commercial law. Identified two image-scanned PDFs (original IPC and old POCSO) with zero extractable text -- replaced them with text-based versions.

```
ACT_MAP = {
    "1742378582_Indian_Penal_Code_1860.pdf":  { "act_name": "Indian Penal Code, 1860",
                                                "short": "IPC", "category": "criminal" },
    "250883_english_01042024.pdf":            { "act_name": "Bharatiya Nyaya Sanhita, 2023",
                                                "short": "BNS", "category": "criminal" },
    "domviolence.pdf":                        { "act_name": "Protection of Women from DV Act, 2005",
                                                "short": "PWDVA", "category": "civil_rights" },
    "NI.pdf":                                 { "act_name": "Negotiable Instruments Act, 1881",
                                                "short": "NIA", "category": "commercial" },
    # ... 13 more acts
}
```

2. PyMuPDF Text Extraction

Used PyMuPDF to extract raw text from each PDF page by page. Applied clean_text() to remove form-feed characters (\x0c), control characters, and excessive whitespace before passing text to the chunker.

```
def extract_text(pdf_path: Path) -> str:
    doc = pymupdf.open(str(pdf_path))
    return clean_text("\n".join(p.get_text() for p in doc))

def clean_text(text: str) -> str:
    text = text.replace("\x0c", "\n")                          # form-feed
    text = re.sub(r"[\x00-\x08\x0b\x0e-\x1f\x7f]", "", text) # control chars
    text = re.sub(r"\n{3,}", "\n\n", text)                     # excess newlines
    text = re.sub(r"[ \t]{2,}", " ", text)                       # extra spaces
    return text.strip()
```

3. Section-Based Chunker

Indian statutes are structured by numbered sections (e.g., "11. Standard rent"). Built a regex boundary detector that splits on section numbers, keeping each legal section as one chunk. Small sections (< 80 words) are merged with the next. Falls back to sliding window if fewer than 3 section boundaries are found.

```
boundary = re.compile(
    r"(?:^|\n)\s*"
    r"(?:Section\s+|Article\s+|Sec\.\s*)?"
    r"(\d{1,3}[A-Z]{0,2})\s*"
    r"[\.---]\s*"
    r"(?=[A-Z\(\[])",
    re.MULTILINE,
)
# Sections < 80 words are merged with the next one
# Result: 200-800 word chunks, each = one complete legal section
```

4. Sliding Window Fallback

For PDFs without clear section headers (textbooks, guides), falls back to overlapping word windows. 600 words per chunk with 60-word overlap ensures context is not lost at chunk boundaries.

```
def sliding_window(text: str, max_words=600, overlap=60):
    words = text.split()
    chunks, start = [], 0
    while start < len(words):
        end = min(start + max_words, len(words))
        chunks.append(" ".join(words[start:end]))
        if end >= len(words): break
        start = end - overlap  # 60-word overlap prevents context loss
    return chunks
```

5. Embedding + ChromaDB Batch Insertion

Each chunk is embedded with the "passage: " prefix (required by multilingual-e5-small for indexed documents), normalized to unit length, then inserted into ChromaDB with act name, short code, category, and chunk index as metadata. Inserted in batches of 100.

```
embeddings = model.encode(
    [f"passage: {s}" for s in sections],
    batch_size=32,
    normalize_embeddings=True,   # L2 norm required for cosine similarity
)
for b in range(0, len(ids), 100):
    col.add(
        ids=ids[b:b+100],
        documents=docs[b:b+100],
        metadatas=metas[b:b+100],
        embeddings=embeds[b:b+100],
    )
```

6. Incremental Topup Script (topup_new3.py)

Built a topup script to add the 3 new acts without dropping and re-ingesting all 17 acts. Chunk IDs are md5(filename_index) so duplicates are impossible. Final collection: 2,255 chunks across 17 acts.

- IPC 1860 (text-based replacement): 367 chunks
- Negotiable Instruments Act 1881: 128 chunks
- Protection of Women from Domestic Violence Act 2005: 25 chunks

Suyash -- Frontend + Chat UI

Developed Next.js 14 frontend with chat UI, Triage Wizard, and real-time streaming

1. Next.js 14 App Router Setup

Set up the Next.js 14 project with App Router, TypeScript, and Tailwind CSS. Configured NEXT_PUBLIC_API_URL environment variable so the backend URL is injectable at build time for both local dev and cloud deployment.

```
// frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:8000

// Used in ChatInterface.tsx
const API = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000"
```

2. SSE Stream Reading (ChatInterface.tsx)

Used the browser Streams API to read SSE response token by token instead of waiting for the full answer. Each line is parsed as a JSON event and dispatched by type: "sources" populates badge list, "token" appends to the message, "done" finalises and stops the blinking cursor.

```
const res = await fetch(`${API}/api/query`, { method:"POST", body: JSON.stringify({...}) })
const reader = res.body!.getReader()
const decoder = new TextDecoder()
while (true) {
  const { done, value } = await reader.read()
  if (done) break
  for (const line of decoder.decode(value).split("\n")) {
    if (!line.startsWith("data: ")) continue
    const ev = JSON.parse(line.slice(6))
    if (ev.type === "sources") setSources(ev.sources)
    if (ev.type === "token")   appendToken(ev.text)
    if (ev.type === "done")    setStreaming(false)
  }
}
```

3. MessageBubble Component (MessageBubble.tsx)

User messages: orange bubble, right-aligned. Assistant messages: white card, left-aligned, source badges above. Loading state: 3 bouncing dots. Streaming state: blinking orange cursor after the last token. Uses react-markdown + remark-gfm so tables, bold, lists render correctly.

```
import ReactMarkdown from "react-markdown"
import remarkGfm from "remark-gfm"

<div className="prose prose-sm ... prose-table:w-full prose-th:bg-orange-50
                prose-td:border prose-td:border-gray-200">
  <ReactMarkdown remarkPlugins={[remarkGfm]}>
    {msg.content}
  </ReactMarkdown>
  {msg.streaming && <span className="blinking-cursor" />}
</div>
```

4. Source Citation Badges

When the SSE "sources" event arrives before tokens start, renders coloured pill badges above the answer showing exactly which acts were retrieved. Gives users transparency about which laws back the answer before they finish reading it.

```
{msg.sources?.map(s => (
  <span key={s}
    className="text-xs bg-blue-100 text-blue-800 border border-blue-200
               rounded-full px-2 py-0.5 font-medium">
    {s}
  </span>
))}
```

5. Triage Wizard (TriageWizard.tsx)

Collapsible sidebar panel with 6 legal categories. Clicking a category expands 3-4 sub-categories. Each sub-category has a pre-written legal prompt and a category tag. Clicking fires the query into the chat with the correct category filter applied -- bypassing the need for the user to type anything.

```
const CATEGORIES = [
  { id: "labour", label: "Labour Rights", subs: [
    { label: "Unpaid Salary",
      prompt: "My employer has not paid my salary for X months...",
      cat: "labour" },
    { label: "Wrongful Termination",
      prompt: "I was fired without notice or reason...",
      cat: "labour" },
  ]},
  { id: "property", label: "Property & Rent", subs: [...] },
  // 4 more categories: Consumer, Criminal, RTI, Civil Rights
]
```

6. Tailwind Typography Plugin Configuration

Installed @tailwindcss/typography and configured prose classes with custom overrides for table styling. Before this fix, LLM-generated markdown tables rendered as raw | pipe | dash | text -- unreadable. After: proper HTML tables with borders, alternating row colours, and orange header tint matching the brand.

```
// tailwind.config.ts
plugins: [require("@tailwindcss/typography")]

// MessageBubble.tsx -- prose overrides
prose-table:w-full prose-table:text-sm
prose-th:bg-orange-50 prose-th:border prose-th:border-gray-300
prose-td:border prose-td:border-gray-200
prose-tr:even:bg-gray-50
```

Avishkar -- RAG Pipeline + Smart Retrieval

Implemented RAG retrieval logic with category auto-detection and cosine similarity tuning

1. multilingual-e5-small Integration

Chose intfloat/multilingual-e5-small as the local embedding model -- small enough to run on CPU without a GPU, supports Hindi and Indian regional languages, and outperforms generic English models on multilingual legal text. Critical discovery: e5 models need DIFFERENT text prefixes for indexing vs retrieval.

```
# Indexing time (ingest.py): "passage: " prefix
model.encode(f"passage: {chunk}", normalize_embeddings=True)

# Query time (rag.py): "query: " prefix
model.encode(f"query: {user_question}", normalize_embeddings=True)

# Using wrong prefix degrades similarity scores significantly.
# normalize_embeddings=True converts to unit vectors for cosine distance.
```

2. ChromaDB Cosine Similarity Setup

Configured ChromaDB with cosine distance metric (hnsw:space="cosine"). At query time, retrieves top-K chunks sorted by similarity score. Converts raw distance to human-readable relevance score for source badges.

```
# Collection created with cosine space
col = client.create_collection(
    COLLECTION,
    metadata={"hnsw:space": "cosine"}
)

# Convert distance to similarity score for display
score = round(1 - distance, 4)   # distance=0 means identical, score=1.0
```

3. Category Auto-Detector (_detect_category)

Solved the critical retrieval bug: without a category filter, BNS and IPC dominate search results by sheer volume (367 chunks each), even for rent or labour queries. Built keyword-based domain router that detects the legal category from the query and applies the ChromaDB "where" filter automatically -- no user action needed.

```
_CATEGORY_KEYWORDS = {
    "property": ["rent", "landlord", "tenant", "1rk", "1bhk",
                 "paying rent", "security deposit", "eviction", ...],
    "labour":   ["salary", "wage", "fired", "dismissed", "epf",
                 "gratuity", "minimum wage", "termination", ...],
    "criminal": ["murder", "theft", "fir", "arrest", "bail", ...],
    "consumer": ["refund", "defective", "consumer", "amazon", ...],
    # 4 more categories
}

def _detect_category(query: str) -> Optional[str]:
    q = query.lower()
    scores = {cat: sum(1 for kw in kws if kw in q)
              for cat, kws in _CATEGORY_KEYWORDS.items()}
    scores = {k: v for k, v in scores.items() if v > 0}
    if not scores: return None
```

```
    best, top = max(scores.items(), key=lambda x: x[1])
    # Only filter if one clear winner -- prevents wrong routing
    return best if list(scores.values()).count(top) == 1 else None
```

4. Retrieve Function with Auto-Routing

The retrieve() function uses category from the Triage Wizard if provided, otherwise calls _detect_category(). When filtered, fetches 8 chunks instead of 6 -- filtered searches cover fewer acts so more chunks per act gives richer legal context.

```
def retrieve(query, k=TOP_K, category=None):
    embedding = model.encode(
        f"query: {query}", normalize_embeddings=True
    ).tolist()

    # Explicit (Triage Wizard) takes priority over auto-detected
    effective_category = category or _detect_category(query)
    where = {"category": effective_category} if effective_category else None

    # Fetch more chunks when filtered (fewer competing acts)
    actual_k = min(k + 2, 10) if effective_category else k

    results = collection.query(
        query_embeddings=[embedding],
        n_results=actual_k,
        where=where,
        include=["documents", "metadatas", "distances"]
    )
```

5. Context Builder + Source Deduplication

Builds the LLM context by numbering each retrieved chunk with its act name. Deduplicates source act names for the SSE "sources" event -- the same act may appear in multiple top-K chunks. Sources are sent BEFORE tokens so the UI shows act badges immediately, before the answer starts streaming.

```
# Deduplicated act names for source badges (sent first)
sources = list({c["act_name"] for c in chunks})
yield f'data: {{"type":"sources","sources":{sources}}}\n\n'

# Numbered context string for LLM
context = "\n\n---\n\n".join(
    f"[Source {i} -- {c['act_name']}]\n{c['text']}"
    for i, c in enumerate(chunks, 1)
)
```

6. Groq Fallback + Token Tuning

Implemented try/except fallback: if gpt-oss-120b is unavailable or rate-limited, automatically retries with gpt-oss-20b. Both models were confirmed by querying the /v1/models endpoint during development. max_tokens raised from 1024 to 2048 after truncation bug was caught mid-sentence ("He **cannot...").

```
try:
    stream = groq.chat.completions.create(
        model="openai/gpt-oss-120b",
        max_tokens=2048,     # raised from 1024 to fix truncation
        temperature=0.1,
```

```
        stream=True, ...)
except Exception:
    stream = groq.chat.completions.create(
        model="openai/gpt-oss-20b",   # fallback model
        max_tokens=2048,
        stream=True, ...)
```

LexAid -- Team Summary

AI-Powered Indian Legal Aid Platform

Snehal -- Backend API + LLM Streaming

FastAPI + Groq SSE + System Prompt + CORS

Suraj -- Data Ingestion + Vector Database

PyMuPDF + ChromaDB + 17 Acts + 2,255 Chunks

Suyash -- Frontend + Chat UI

Next.js 14 + Triage Wizard + react-markdown + remark-gfm

Avishkar -- RAG Pipeline + Smart Retrieval

multilingual-e5-small + Category Auto-Detect + Cosine Tuning

Complete Tech Stack

Backend:	FastAPI 0.115 | Python 3.10 | Uvicorn
AI / LLM:	Groq gpt-oss-120b | multilingual-e5-small (local)
Database:	ChromaDB 0.5 | Supabase Postgres (planned)
Frontend:	Next.js 14 | TypeScript | Tailwind CSS
Libraries:	PyMuPDF | sentence-transformers | react-markdown | remark-gfm
Corpus:	17 Indian Acts | 2,255 vector chunks | Cosine similarity
Deployment:	Vercel (frontend) | Railway/Render (backend) -- Planned Phase 2